

CloudStock — Relatório completo do projeto

Data do documento: 09 de outubro de 2026

Autor do projeto: Keyllin (usuário GitHub: seukeyllin)

Repositório: <https://github.com/seukeyllin/cloudstockproject>

Natureza: Trabalho de Conclusão de Curso (TCC) — Desenvolvimento de Sistemas

Nome do aplicativo: CloudStock

1. Descrição total do projeto

O CloudStock é um gerenciador de ROMs e backups de jogos retrô, pensado como um front-end com visual de loja digital (referência explícita: Nintendo eShop). O objetivo acadêmico e prático é organizar, visualizar e sincronizar uma biblioteca de jogos de várias plataformas, com cara de produto — e não de pasta solta de arquivos.

O sistema foi concebido para:

- mostrar os jogos em grade (capas, título, plataforma);
- filtrar por sistema (NES, N64, PlayStation 1, PlayStation 2 e “todos”);
- permitir tema claro e tema escuro;
- escanear pastas reais de ROMs e montar a biblioteca automaticamente;
- aceitar metadados manuais para jogos com nomes estranhos, hacks e variações;
- guardar capas e mídia em uma estrutura inspirada no ES-DE (EmulationStation Desktop Edition);
- persistir a pasta escolhida entre sessões;
- no uso final, ler e fazer backup dos arquivos a partir da nuvem (Mega primeiro, Google Drive depois), e não apenas do disco local;
- reaproveitar o máximo possível do código em uma versão Android futura;
- no médio prazo, vincular pastas de emuladores, baixar mídia de fontes especializadas e, depois, suportar navegação por gamepad.

Stack escolhida

Camada	Tecnologia
Desktop	Tauri 2 (Rust + WebView)
Front-end	Vite + Svelte + TypeScript
Controle de versão	Git + GitHub
Editor	VSCodium (alternativa gratuita ao VS Code)
Persistência local	@tauri-apps/plugin-store
Seletor de pasta	@tauri-apps/plugin-dialog
Leitura de arquivos	@tauri-apps/plugin-fs
Nuvem (iniciada)	megajs (SDK não oficial do Mega)

A escolha do Tauri, em vez de Electron, foi deliberada: app mais leve e possibilidade real de reutilizar a interface web numa versão Android sem reescrever tudo do zero.

Modelo de dados previsto

Cada jogo é um objeto `Rom` com, no mínimo:

- identificador (nome do arquivo);
- título;
- plataforma;

- caminho do arquivo;
- caminho da capa;
- tamanho e data;
- campos opcionais de metadado: ano, coleção e descrição.

Os metadados manuais não ficam um arquivo por jogo. A regra definida pelo usuário é **um único arquivo por sistema**, chamado `cloudstore.metadata.txt`, dentro da pasta daquele sistema. O formato acordado é em blocos:

```
game: superdiasland(EUA).rar
title: Super Dias Land
system: n64
year: 2004
collection: Super Dias
description: Um jogo eletrizante com gráficos 2D[...]
```

A mídia visual segue a lógica do ES-DE, com a pasta `covers` **dentro de cada sistema** (decisão do usuário, melhor que uma pasta única de capas):

```
downloaded_media/
├── nes/covers/
├── n64/covers/
├── ps1/covers/
└── ps2/covers/

src/assets/
├── icons/
├── themes/
└── logos/
```

Ponto importante de produto: o usuário deixou claro, depois do scan local funcionar, que a pasta “de verdade” do uso final deve ser a do Mega ou do Drive. O scan local passou a ser ferramenta de desenvolvimento e fallback, não o destino final do TCC.

2. O que o usuário queria e ainda quer implementar

Já pedido e ainda em aberto

- Login no Mega que realmente autentique e mantenha a sessão.
- Listar os jogos que estão na nuvem e mostrá-los na página inicial, sem cadastrar jogo por jogo.
- Reconhecimento de um jogo mesmo com variação de nome de arquivo, desde que não seja um nome completamente diferente.
- Arquivo único `cloudstore.metadata.txt` por sistema, para hacks e nomes fora do padrão.
- Backup na nuvem (botão “Backup na Nuvem” existe na interface, sem lógica).
- Botão “Adicionar ROM” (existe na interface, sem lógica).
- Capas reais carregadas de `downloaded_media/{sistema}/covers/`, com fallback quando a imagem não existe.
- Integração com Google Drive (pedido, não iniciado).
- Download automático de capas, descrições e mídia de sites especializados, aproveitando a estrutura estilo ES-DE.
- Vínculo com as pastas dos emuladores e organização automática (“já baixaria tudo nas pastas corretas”).
- Tema claro/escuro já existe; o usuário também falou em botão no canto superior esquerdo — o botão ficou no header, ao lado do logo.
- Suporte a gamepad, planejado para depois da navegação por mouse e teclado.
- Versão Android reaproveitando o código do desktop (motivo da escolha do Tauri).
- Trabalhar nos dois computadores (casa e escola) no mesmo repositório.

Restrições reais do ambiente

- PC de casa: com permissão de administrador.
- PC da escola: sem administrador. Node.js só na versão portátil .zip. Terminal padrão do Git Bash (MINGW64), o que complicou o PATH.

- Preferência de trabalho: alterações cirúrgicas no código (trocar só o bloco que muda), não reescrever o arquivo inteiro a cada passo.

3. Implementações bem-sucedidas

Confirmadas pelo próprio usuário ao longo do desenvolvimento:

1. **Repositório GitHub** cloudstockproject, com README, licença MIT e .gitignore de Node.
2. **Ambiente de casa:** VSCodium, Node.js, Rust/Tauri, WebView e build tools. A janela do Tauri abriu com o nome do projeto.
3. **Primeiro commit** e limpeza do template padrão do Vite/Svelte.
4. **Estrutura de pastas do front-end** (components, layouts, pages, stores, utils, types, assets — conforme o plano inicial) e, depois, pastas de mídia e assets.
5. **Layout estilo eShop:** header, sidebar de plataformas, grade de cards, contador de ROMs.
6. **Filtro por plataforma** funcionando (“Tema e filtro funcionando”).
7. **Alternância claro/escuro** funcionando.
8. **Tipos TypeScript** (Rom) atualizados para metadados (ano, coleção, descrição, caminho, tamanho).
9. **Store de ROMs** (romsStore) ligada à interface (“App conectado ao store”).
10. **Parser de metadados** (metadataParser.ts) para um cloudstore.metadata.txt por pasta de sistema, inclusive aceitando o tipo description.
11. **Scan local** de pastas por extensão de arquivo (“Scan funcionando”), com mapa de extensões para NES, N64, PS1 e PS2.
12. **Seletor de pasta** do sistema (“Seletor abriu”), depois de registrar o plugin com `npx tauri add dialog`.
13. **Persistência da pasta escolhida** (“Pasta salva e carregada automaticamente”), com `@tauri-apps/plugin-store`.
14. **Serviço Mega** criado em `src/stores/mega.ts` (loginMega, logoutMega, stores de sessão), usando `megajs` e `await storage.ready`.
15. **Botão e modal de login do Mega** desenhados na interface (email, senha, cancelar, entrar, estado “Mega Conectado”).

Ajustes de produto feitos pelo usuário e incorporados:

- Capas dentro de cada sistema, não numa pasta global.
- Sidebar com `<button>` em vez de `<a href="#">`.
- Um único arquivo de metadados por sistema, não um .txt por ROM.
- Melhorias visuais próprias no CSS (cards, botão verde de scan, tema claro).

4. Implementações mal sucedidas ou incompletas

Nada foi “jogado fora” de forma irreversível, mas várias tentativas falharam ou pararam no meio:

Tentativa	O que aconteceu
Node portátil no PC da escola	Várias falhas de PATH. Comandos de PowerShell foram colados no Git Bash (<code>\$env:PATH</code> , <code>command not found</code>). O ls não encontrava Desktop/Documents pelo caminho <code>/c/Users/...</code> . A pasta do Node foi movida mais de uma vez (Desktop, Documents, <code>C:\CloudStock\...</code>). Não há confirmação final de que o Tauri rodou na escola.
Plugin “local-storage”	<code>npx tauri add local-storage</code> falhou: o crate não existe. O plugin correto é store.
Seletor de pasta na primeira tentativa	<code>npm install @tauri-apps/plugin-dialog</code> sozinho não bastou. O diálogo só abriu depois de <code>npx tauri add dialog</code> .

Tentativa	O que aconteceu
Array fixo de jogos	Serviu só de demonstração. O usuário rejeitou com razão: cadastrar jogo por jogo é inviável.
onerror da capa em HTML puro	Svelte/TypeScript recusou string de evento e depois recusou acesso a .style sem cast. O fallback de imagem ficou instável.
HTML do App.svelte ao inserir o modal	Tags </div> a mais/a menos geraram “invalid closing tag” e erros de compilação.
Acessibilidade do modal Mega	Warnings persistentes do Svelte (role="dialog" precisa de tabindex; div com clique precisa de papel ARIA e eventos de teclado). Foram aplicados role, aria-modal e tabindex="-1". O último aviso de clique em elemento não interativo ainda aparecia na última captura.
Login Mega de ponta a ponta	A UI e a store existem. Não há confirmação de que um login real no Mega.nz funcionou, nem de listagem de arquivos da conta.
Google Drive Backup, adicionar ROM, gamepad, Android, scrape de mídia, vínculo com emulador	Não iniciado.
Capas reais na grade	Botões ou ideias existem; a lógica não.
	O caminho coverPath foi preparado, mas o carregamento confiável de imagem (e o protocolo de arquivo local do Tauri) não foi fechado.

5. Histórico de avanço

Ordem aproximada, do início do chat dedicado ao código até a parada.

1. Decisão de usar este chat só para o código do TCC.
2. Criação do repositório GitHub, com dúvida sobre licença e .gitignore antes de escolher a linguagem. Foram adotados MIT e template Node.
3. Escolha do VSCode (casa com admin; escola sem admin).
4. Clone do repositório e popup “Do you trust the authors”.
5. Estudo do EmulationStation como referência (frontend gráfico, metadados, lançamento de emulador) e discussão de fontes externas de download. Ficou registrado o interesse em FitGirl-like e login Drive/Mega; fontes de ROM pirateada não viraram funcionalidade implementada.
6. Requisito de versão Android e reúso de código → escolha do **Tauri** em vez de Electron.
7. Erros de ambiente em casa: npm não reconhecido; política de execução do PowerShell; diretório não vazio no Vite; script tauri ausente no package.json.
8. Janela Tauri abre pela primeira vez.
9. Tentativa longa de replicar o ambiente na escola (Node .zip, Git Bash, export PATH). O trabalho volta para o PC de casa.
10. Primeiro commit, limpeza do template e estrutura de pastas.
11. Layout eShop, filtro e tema — confirmados com captura de tela.
12. Pastas downloaded_media e src/assets.
13. Abandono do cadastro manual: tipos, store, parser de metadados e scan por extensão.
14. Scan local confirmado. Seletor de pasta. Persistência da pasta.
15. Correção explícita do usuário: a origem final dos arquivos é a nuvem.
16. Início do Mega: npm install megajs, src/stores/mega.ts, botão e modal.
17. Sequência de erros de markup e de acessibilidade no App.svelte. O desenvolvimento para aqui, sem login Mega confirmado e sem listagem na nuvem.

6. Fases do projeto e onde parou

Fase	Conteúdo	Estado em 09/10/2026
0. Definição	TCC, eShop, backups, emuladores, Android, nuvem	Concluída como visão
1. Fundação	GitHub, editor, Node, Tauri, Svelte	Concluída no PC de casa.
2. Casca visual	Layout, tema, filtro, cards	Escola incompleta
3. Biblioteca local	Tipos, store, metadados, scan, seletor, pasta salva	Concluída
4. Conta Mega	SDK, store, modal, botão de sessão	Concluída para uso local de teste
5. Biblioteca na nuvem	Listar Mega (e depois Drive) e popular a grade	Parou aqui. UI pronta; login real não validado
6. Backup e escrita	Enviar/organizar ROMs na nuvem e nas pastas dos emuladores	Não iniciada
7. Mídia automática	Baixar capas/descrições para downloaded_media	Não iniciada
8. Produto de TCC	Gamepad, polimento, Android, documentação acadêmica	Não iniciada

Fase atual: fim da fase 4, início não dado da fase 5.

O aplicativo já é um front-end local demonstrável. Ainda não é o organizador de nuvem que o TCC descreve.

7. Ideias de próximas implementações e como chegar no resultado

Ordem recomendada, da mais bloqueante para a mais tardia.

7.1 Fechar o modal e o login Mega

- Tratar o aviso restante de acessibilidade: no overlay, usar `on:click` junto com `on:keydown` (Escape fecha) ou um `<button>` invisível de fechar, para o Svelte parar de reclamar de elemento não interativo.
- Testar `loginMega` com uma conta real e ler o erro cru no console (o alert genérico esconde a causa: 2FA, e-mail errado, bloqueio do WebView, crypto do megajs no browser).
- Se o megajs falhar dentro do WebView do Tauri, mover o login para um comando Rust ou isolar a chamada e tratar cookies/CORS. Não salvar senha em texto puro no `settings.json`. No máximo um token de sessão, se a biblioteca expuser, e com aviso na interface.

7.2 Listar arquivos do Mega e encaixar no `romsStore`

- Depois do `storage.ready`, percorrer as pastas da conta (ou uma pasta raiz escolhida, por exemplo `CloudStock/nes`).
- Reutilizar o mapa de extensões que já existe no scan local.

- Para cada arquivo, procurar o bloco correspondente em `cloudstore.metadata.txt` (esse `.txt` pode viver na mesma pasta do Mega ou numa cópia local em cache).
- Preencher `Rom.title` com o metadado; se não houver, usar o nome do arquivo limpo (trocar `_` e `-` por espaço, tirar região entre parênteses só como `*fallback*`, nunca como única regra).
- Matching “parecido, mas não idêntico”: normalizar (minúsculas, sem acento, sem (USA) / [!]) e comparar o nome do arquivo com o campo `game:` e com o `title`. Hacks e nomes absurdos continuam dependendo do `.txt` — isso já é a regra certa e não deve ser substituída por um chute automático agressivo.

7.3 Capas

- Convenção estável: `downloaded_media/{plataforma}/covers/{id-normalizado}.jpg`.
- No Tauri, arquivo local não deve ir cru no `src` da `<img>`. Usar o protocolo de asset do Tauri (`convertFileSrc`) ou copiar a capa para uma pasta servida.
- `on:error` com `cast` para `HTMLImageElement` (já tentado) ou, mais simples, um componente `Svelte Cover.svelte` com estado `failed`.

7.4 Drive

- Só depois do Mega listar de verdade.
- Google Drive exige OAuth (client ID, redirect, escopo `drive.readonly` no início). No Tauri isso costuma ser um plugin ou um fluxo no navegador do sistema + token guardado no `plugin-store`.
- A interface já pode ter dois estados de origem: `local` | `mega` | `drive`. A grade não precisa saber a diferença, só o `Rom`.

7.5 Backup

- O botão de backup, no desenho atual, deve copiar da origem escolhida para o destino (Mega ↔ disco, ou Drive ↔ disco), com fila e progresso.
- Não prometer “baixar site de ROM” dentro do app. O fluxo acadêmico defensável é: o usuário já tem os arquivos na nuvem dele; o CloudStock organiza, descreve, mostra capa e sincroniza.

7.6 Mídia automática

- Serviço separado que, dado `title` + `platform` + `year`, consulta uma API de metadados de jogos (não um site de download de ROM) e grava em `downloaded_media/{sistema}/covers/`.
- Cache local para não baixar a mesma capa duas vezes.
- Se a API não achar, o `cloudstore.metadata.txt` continua mandando.

7.7 Pastas de emuladores

- Um mapa configurável: `nes` → `C:\Emulators\Mesen\roms`, etc.
- “Adicionar ROM” e o backup passam a copiar para essa pasta além da nuvem.
- Lançar o emulador pode ser um comando Tauri (`Command`) numa fase posterior; não é necessário para o núcleo do TCC.

7.8 Gamepad e Android

- Gamepad: depois que toda ação importante for um botão focável (já é o caso da sidebar). Uma lib de gamepad ou a Gamepad API do browser, com foco visual na grade.
- Android: o Tauri 2 gera o projeto móvel a partir do mesmo front. O que **não** reaproveita sozinho é diálogo de pasta do Windows, caminhos `C:\` e plugins nativos. A lógica de store, tipos, parser e UI Svelte reaproveita.

7.9 Escola

- Não insistir em Git Bash para Node. Abrir o terminal do VSCodium como PowerShell, ou colocar o `node.exe` portátil numa pasta fixa e um script `.bat` que só ajusta o PATH da sessão.
- `git pull` / `git push` mantêm casa e escola iguais. Não duplicar lógica “versão escola”.

7.10 O que falta para o resultado necessário do TCC

O mínimo demonstrável e honesto:

1. Abrir o app e ver a grade estilo loja.
 2. Conectar o Mega (ou mostrar o Drive) e listar os jogos da conta, não um array escrito à mão.
 3. Aplicar `cloudstore.metadata.txt` em pelo menos um hack e um jogo com nome sujo.
 4. Mostrar capa quando o arquivo existir e um placeholder quando não existir.
 5. Filtrar por plataforma e alternar tema.
 6. Lembrar a origem (pasta ou conta) na próxima abertura.
 7. Um fluxo de backup com começo, progresso e fim, mesmo que simples.
 8. Texto acadêmico alinhado com o que o código realmente faz.
-

8. Resumo geral

O CloudStock é o TCC de um organizador de ROMs retrô com interface de eShop, feito em Tauri, Svelte e TypeScript, com repositório público [seukeyllin/cloudstockproject](https://github.com/seukeyllin/cloudstockproject). A visão é uma biblioteca que se organiza sozinha, lê metadados flexíveis, guarda capas no estilo ES-DE e, no uso real, vive na nuvem (Mega e depois Google Drive), com reuso futuro no Android.

Até 09/10/2026 o projeto **passou da prova de conceito visual e da biblioteca local**. Tema, filtro, tipos, parser de um `cloudstore.metadata.txt` por sistema, scan por extensão, seletor de pasta e gravação dessa pasta estão feitos e foram confirmados. A estrutura de pastas de mídia também está criada.

O projeto **parou na porta da nuvem**. O código de login Mega e o modal existem, mas o último estado conhecido ainda era correção de markup e de avisos de acessibilidade. Não há listagem de arquivos do Mega, não há Drive, não há backup de verdade, não há download de mídia e não há Android. O PC da escola ficou como ambiente incompleto por causa do Node portátil e do Git Bash.

O caminho curto até um TCC apresentável é: validar o login Mega, reutilizar o scan/metadados que já existem para montar a grade a partir da conta, fechar as capas, e só então partir para Drive e backup. O scan local deve permanecer como modo de teste, não como substituto da nuvem que o usuário definiu como requisito central.